

Unit 2 — Data Transportation & Analysing Streaming Data

Syllabus: message queue · core concepts · security · applying core concepts to business logic · in-flight data analysis · distributed stream-processing architecture · key features of stream-processing frameworks

Sources: Psaltis *Streaming Data* Ch.3 (printed pp.38–57) and Ch.4 (pp.58–76) + all question papers. **BOOK p.xx** ↗ opens *Streaming-Data[1].pdf* at printed page xx (PDF page = printed + 20). Blue **highlight** = must-remember. **IMPORTANT** = asked before.

0 · Hot-list for Unit 2 (Q3/Q4 — choose one, 16 marks)

Rank	Topic	Asked in
1	Need for message queuing tier	Model QP 1.3 & 3a · CIE-1 Dec'24 2b · CIE-2 Dec'25 1a
2	Producer–broker–consumer diagram & working	SEE Feb'26 4b (6M) · CIE-2 Dec'25 2a (10M)
3	Durable messaging / offline consumers / replay scenario	CIE-2 Dec'25 1b · CIE-1 Dec'24 5b · Model QP 3b
4	Delivery semantics (+ exactly-once)	CIE-1 Dec'24 A5 · CIE-2 Dec'25 A1 · Model QP 4b · SEE Feb'26 3a(iv), 1.5
5	Broker crash / fault tolerance in MQ tier	CIE-1 Dec'24 A4 · CIE-2 Dec'25 A2 · CIE-1 Nov'25 3a · SEE Feb'26 3b
6	Points of failure in stream processing (7 points)	Model QP 4a · CIE-2 Jan'25 3a · SEE Feb'26 8b · CIE-1 Dec'24 4a · CIE-1 Nov'25 3a
7	Stream-processing frameworks (Storm, Samza, Flink, Spark) & architecture	Model QP 1.4 · CIE-2 Jan'25 A1,A2,2a · CIE-3 Jan'26 A1,1a,4 · SEE Feb'26 1.4 (Flink), 4a (Samza)
8	State management; state-machine vs rollback recovery	CIE-3 Jan'26 2a,3b · CIE-2 Jan'25 A3
9	Traditional DBMS vs streaming (continuous query)	CIE-2 Jan'25 1a · CIE-3 Jan'26 A2
10	Scenario tables (Coke machines, traffic replay, fraud)	SEE Feb'26 3a · Model QP 3b

1 · Why we need a message queuing tier BOOK p.39 ↗ IMPORTANT Model QP

1.3, 3a (10M) · CIE-1 Dec'24 2b (4M) · CIE-2 Dec'25 1a (6M)

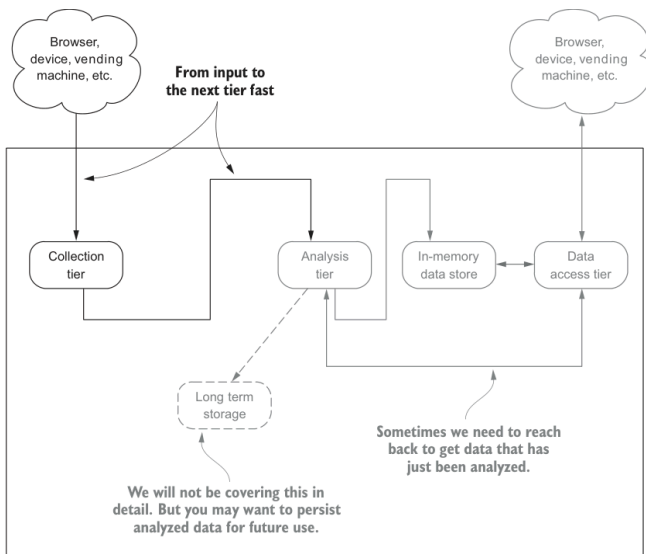


Figure 3.2 From the collection tier straight to the analysis tier

Fig 3.2 Architecture WITHOUT the queue tier (p.40)

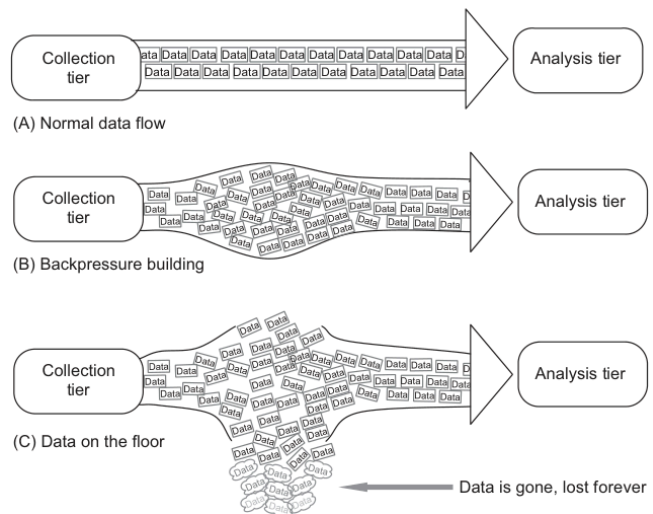


Figure 3.6 The three stages of data flowing without a message queue. We don't want step C.

Fig 3.6 (A) normal → (B) back-pressure builds → (C) data on the floor (p.43)

- Without it, collection calls analysis directly. A single machine may work, but real systems span machines: we want to **decouple the tiers** (and components in each tier). The queue lets tiers work at a higher level of abstraction — pass messages, no explicit calls to the next layer. BOOK p.40 ↗
- Needs / benefits (write 5 points):**
 - Decoupling** of collection from analysis (independent scale, deploy, fail).
 - Buffering / absorbing surges** — producer faster than consumer causes back-pressure; without a queue data is dropped "on the floor" (Fig 3.6C). BOOK p.43 ↗
 - Offline / slow consumers** (e.g. hourly batch readers) — needs durable messaging. BOOK p.44 ↗
 - Durability & replay** / disaster recovery (data-centre link cut).
 - Delivery guarantees, ordering, flow control, fault tolerance** (CIE-2 Dec'25 solution adds FIFO ordering, back-pressure management).
- Example: collection in San Diego, analysis in Amsterdam; fibre cut for 2–3 days — a durable queue keeps the messages (Figs 3.7–3.9). BOOK p.44 ↗

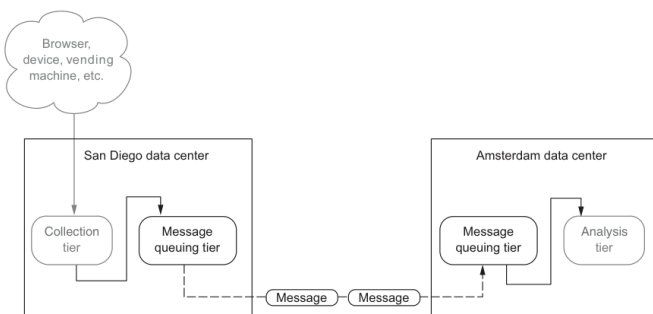


Figure 3.7 Two data centres with data flowing between them

Fig 3.7 Two data centres (p.44)

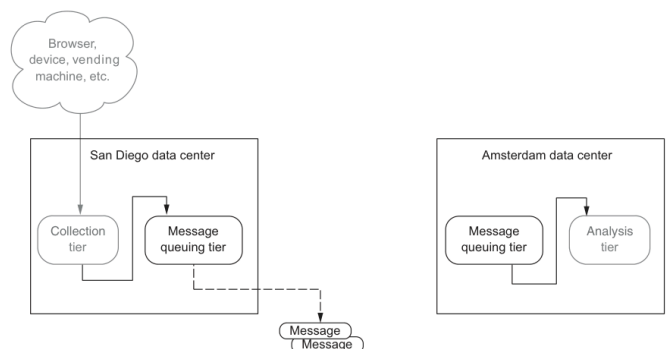


Figure 3.8 Link cut – data flows into the ocean

Fig 3.8 Link cut – data flows into the ocean (p.45)

🔥 HOW TO WRITE IT — Model QP 3a (10M) "elaborate the need for message queuing tier with example"

Intro (Fig 3.1/3.2 without queue) → Fig 3.6 back-pressure story → 5 needs above → San Diego/Amsterdam example → conclusion: "the queue tier lets the collection tier and analysis tier run at their own speed and fail independently".

2 · Core concepts [BOOK p.40 ↗](#)

"Message queuing" is used broadly: RabbitMQ, ActiveMQ, HornetQ (traditional) to NSQ, ZeroMQ and [Apache Kafka](#) (publish/subscribe to streams of records).

2.1 Producer, broker, consumer [BOOK p.41 ↗](#) **IMPORTANT** SEE Feb'26 4b (6M) · CIE-2 Dec'25 2a (10M)

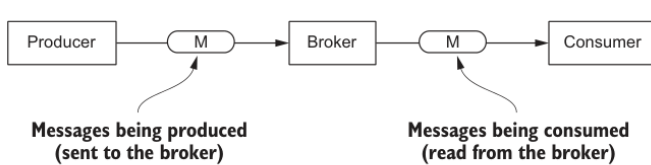


Figure 3.3 The three core parts to a message queuing system

Fig 3.3 The three parts (p.41)

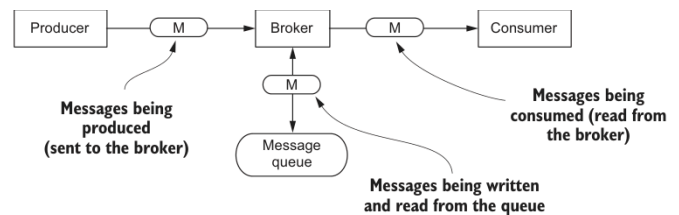


Figure 3.4 The broker with the message queue being shown

Fig 3.4 Broker with the queue shown (p.42)

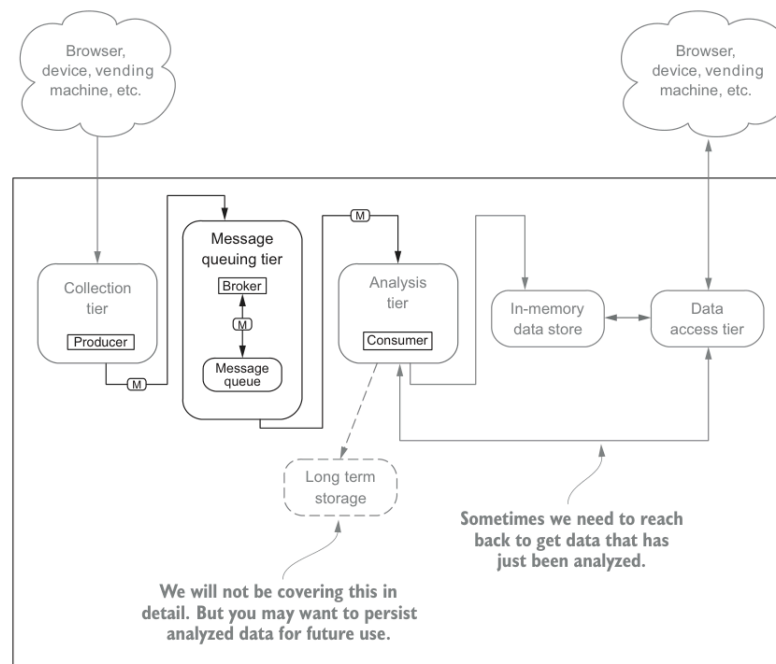


Figure 3.5 Streaming architecture with message queuing tier components shown in context

Fig 3.5 Queue tier components mapped onto the streaming architecture (p.42) — DRAW THIS

1. **Producer** (usually the collection tier) **sends a message to a broker**.
2. **Broker** **puts the message into a queue**. "Broker" not "queue" because the broker is an **abstraction** that may manage **multiple queues**.
3. **Consumer** (usually the analysis tier) **reads the message from the broker**.

🔥 HOW TO WRITE IT — 6M / 10M "working of producer, broker and consumer"

Fig 3.5 + the 3 steps + role of each + then add: durability (queue persists), delivery semantics (ack), decoupling and where each maps (producer = collection tier, consumer = analysis tier). For 10 marks add the failure points of Fig 3.12 (§4) and security of Fig 3.14.

2.2 Isolating producers from consumers [BOOK p.43](#)

- If the producer outruns the consumer (analysis is heavier than collection) pressure builds "like a plugged hose" → data lost (Fig 3.6). It is **not a consumer problem**: consumers may legitimately read slowly or be offline (hourly batch).
- Not all queue products offer producer flow control — then you must throttle the collection tier yourself. Durable messaging is what supports slow/offline consumers.

2.3 Durable messaging [BOOK p.44](#) **IMPORTANT** CIE-2 Dec'25 1b (4M) · CIE-1 Dec'24 5b · Model QP 3b

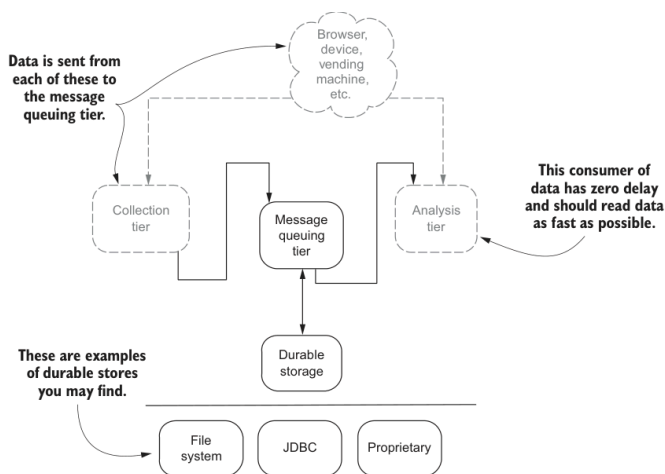


Figure 3.9 Durable messages—where they fit and how they may be stored

Fig 3.9 Durable messages and how they may be stored: JDBC, file system, proprietary (p.46)

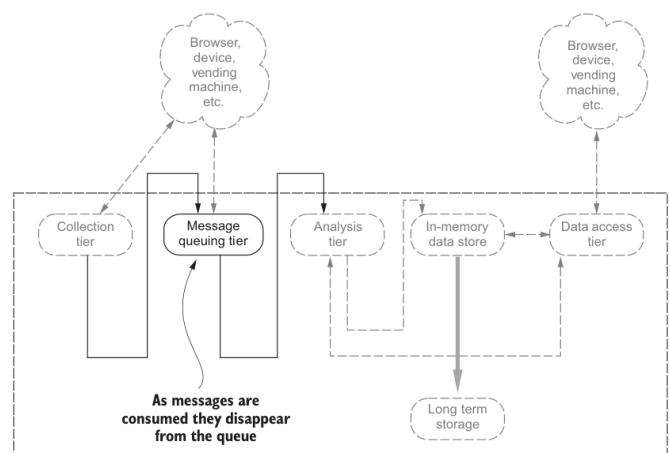


Figure 3.10 Transient messages get discarded after the analysis tier consumes them.

Fig 3.10 Transient messages vanish after consumption (p.46)

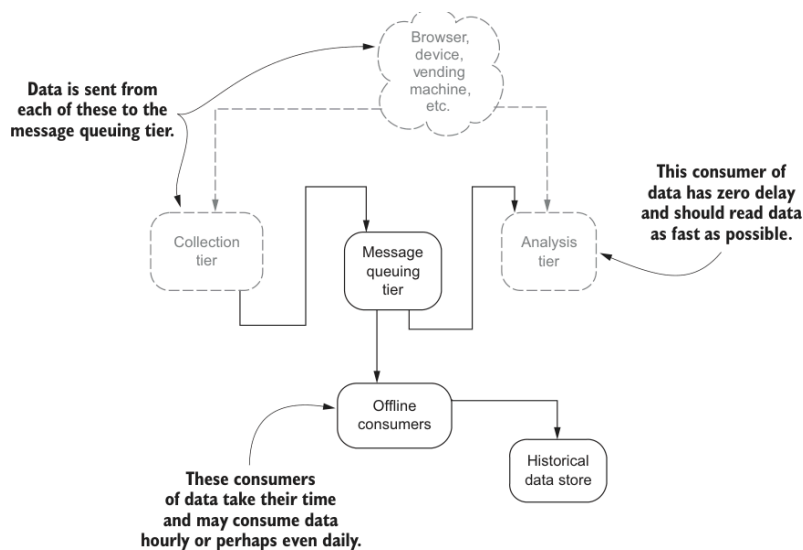


Figure 3.11 Offline consumers persisting data for historical reporting/analysis

Fig 3.11 Offline consumers persist data for historical reporting (p.47)

- **Durable** = messages are **persisted** (database via JDBC, file system, proprietary store) and retained until consumed (or longer), so they survive broker crash, outage and slow/offline consumers. **Transient** = deleted once consumed.

- **Key aspects (CIE-2 Dec'25 scheme):** persistence · message queues marked durable (RabbitMQ, Kafka, JMS) · fault tolerance/resilience · acknowledgements before deletion.
- **Traffic-replay scenario (Model QP 3b / CIE-1 Dec'24 5b style):** app re-routes drivers in real time; 3 months later business wants a *historical traffic-replay product*. With transient queue (Fig 3.10) consumed messages are gone ⇒ impossible. **Solution = Fig 3.11:** durable queue + **offline consumer** that reads at its own pace and writes to a **historical data store**; replay by re-reading retained messages for city/day/week/month. [BOOK p.45 ↗](#)

🗨️ **MNEMONIC — "DOR" — Durable = Outage-proof, Offline consumers, Replay**

2.4 Message delivery semantics [BOOK p.47 ↗](#) **IMPORTANT** CIE-1 Dec'24 A5 · CIE-2 Dec'25 A1 · Model

QP 4b · SEE Feb'26 3a(iv)

Semantics	Meaning (queue tier, Ch.3)	Complexity / speed
At-most-once	May get lost , never re-read	Least complexity, fastest, weakest
At-least-once	Never lost, may be re-read (duplicates)	Medium; consumer must tolerate duplicates
Exactly-once	Never lost, read once and only once	Most complexity, performance hit, strongest (Table 3.1)

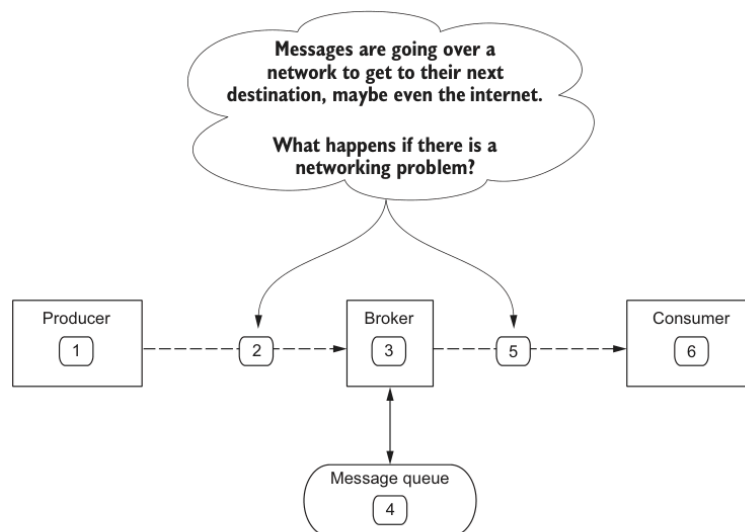


Figure 3.12 Possible points of failure that need to be considered

Fig 3.12 Possible points of failure (p.48)

Six failure points in the queue path (Fig 3.12) [BOOK p.48 ↗](#)

1. **Producer** fails after producing but before sending ⇒ message lost; fails waiting for ack ⇒ resends ⇒ duplicate.
2. **Network producer→broker**: message or ack lost ⇒ producer may resend ⇒ duplicate.
3. **Broker**: crash with messages only in memory ⇒ loss; fails before ack ⇒ resend; fails before committing consumer-read info ⇒ consumer re-reads.
4. **Message queue** (persistent store) fails to write to disk ⇒ loss.
5. **Network broker→consumer**: message sent but not received, or ack lost ⇒ message sent again.

6. **Consumer** fails before recording it processed $\Rightarrow$ re-requests the same message (also duplicates with multiple consumers).

Getting exactly-once when the product doesn't give it [BOOK p.49](#)

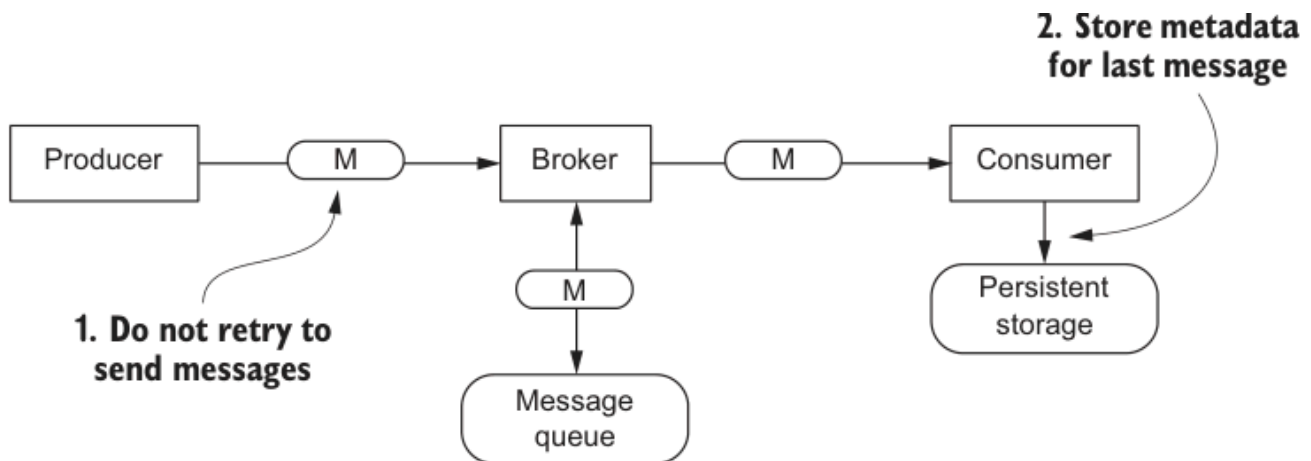


Figure 3.13 The two ways to have exactly-once semantics if the messaging system doesn't provide it

Fig 3.13 Two ways to have exactly-once semantics (p.50)

1. **Do not retry to send messages** — track what the producer sent; if no response, **read the broker to verify** whether the message arrived instead of blindly resending.
2. **Store metadata for the last message** read (JMS: *JMSMessageID*; Kafka: the **offset**) in persistent storage so the consumer never reprocesses.

Bonuses [BOOK p.50](#): **message auditing** (track sent/arrived counts, max/min/avg processing time, find slow producer/consumer) and **duplicate detection** (check metadata/payload identity). **IMPORTANT** SEE Feb'26 1.5: "How do message auditing and duplicate detection improve data quality?" $\rightarrow$ auditing shows lost/late/slow messages; duplicate detection removes repeats $\Rightarrow$ complete, unique, trustworthy data.

⚠ **CONFLICT / TRAP — Does Kafka give exactly-once?**

The textbook (Ch.3, p.49) says products like **Apache Kafka and ActiveMQ do not provide exactly-once** (written 2017). The Kafka book used for Unit 5 (2nd ed., 2021, Ch.8/14) says Kafka supports exactly-once with an **idempotent producer + transactions**, and Kafka Streams enables it with `processing.guarantee=exactly_once`. **Write:** for a generic queue question use the two-technique method above; if the question is about Kafka, say "at-least-once by default; exactly-once via idempotence + transactions".

⚠ **CONFLICT / TRAP — "Six" vs "seven" points of failure**

Ch.3 / Fig 3.12 lists **6** failure points in *message queuing* (producer, 2 networks, broker, queue, consumer). **Ch.4 / Fig 4.15** lists **7** points in *stream processing* (§7). Model QP 4a says "seven points of failure in stream processing data flow" $\Rightarrow$ use the Ch.4 seven. If the question says "queue/delivery semantics failure scenarios" use the six.

🔥 HOW TO WRITE IT — Model QP 4b — "apply delivery semantics to the traffic-replay system, with a neat diagram"

Navigation app + historical replay ⇒ no message may be lost (history must be complete) but duplicates could skew counts ⇒ choose **exactly-once** (or at-least-once + de-duplication). Draw Fig 3.13: producer (no blind retry, verify with broker) → broker/queue → consumer storing offset/metadata in persistent store; mention auditing + duplicate detection bonuses and the trade-off (complexity vs guarantee, Table 3.1).

3 · Security of the queue tier [BOOK p.51](#)

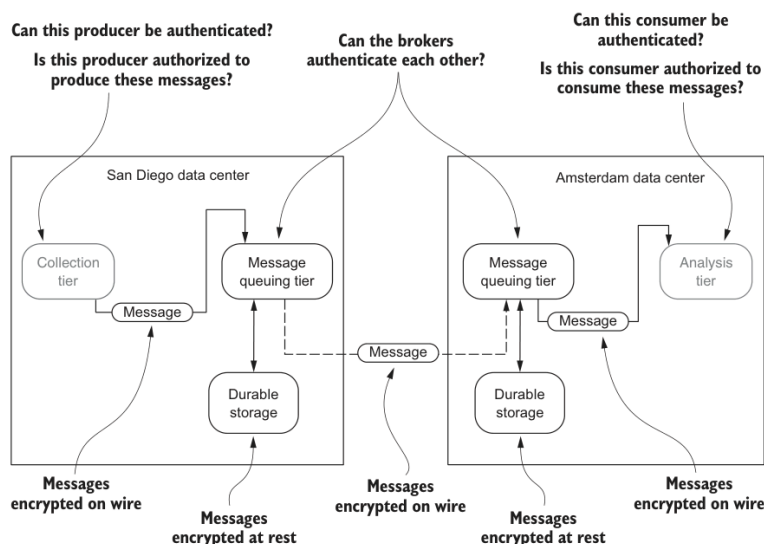


Figure 3.14 Security points that need to be considered for the message queuing tier

Fig 3.14 Security points for the message queuing tier (p.51)

- Messages **encrypted on the wire** (between tiers, brokers, data centres) and **encrypted at rest** (durable storage).
- **Authenticate + authorize producers** ("can this producer be authenticated? may it produce these messages?"), **authenticate + authorize consumers**, and **brokers must authenticate each other**.

4 · Fault tolerance of the queue tier

BOOK p.52

IMPORTANT

SEE Feb'26 3b (8M)

CIE-2 Dec'25 A2 · CIE-1 Dec'24 A4 · CIE-1 Nov'25 3a

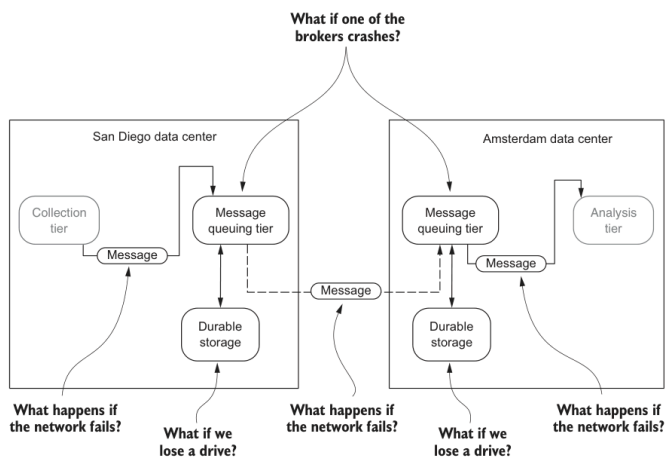


Figure 3.15 A high-level view of where things can go wrong

Fig 3.15 High-level where things can go wrong (p.52)

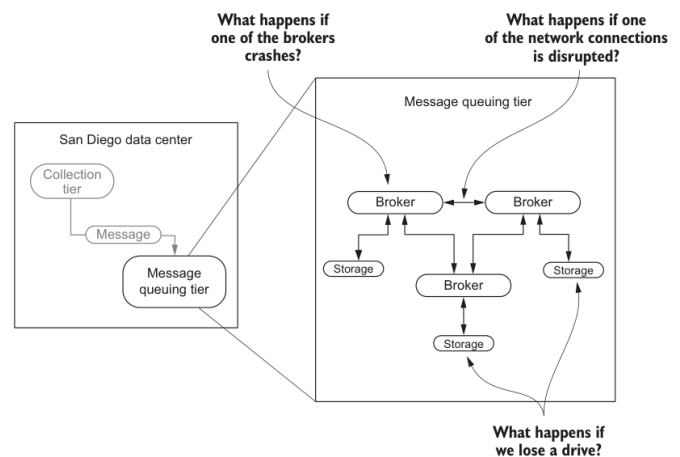


Figure 3.16 Zoomed in view of brokers and what can go wrong

Fig 3.16 Zoom on brokers (p.53)

Failure	Impact on reliability	Mitigation (book p.53–54)
Broker crashes	With durable storage only in-memory messages at risk; messages unacknowledged are lost / re-sent	(1) producer waits for ack that data was written to disk; (2) replicate to > 1 broker (two must crash together); (3) keep as little in memory as possible (trade performance for durability)
Network between brokers is interrupted	With replication data is safe on other brokers; broker later rejoins and synchronises missed messages . Risk: producer mid-send to the severed broker	Ask: is a new replica chosen? what if connection restored? configurable delay before "down"? what if never restored?
Storage drive fails	Potentially drastic; data in flight/not yet on disk lost	Replicas of the lost data? is un-flushed replicating data lost? how to recover the broker?

Also covered by chapter 2 lessons: producer handles network failures; durable store mitigates inter-data-centre link loss.

5 · Applying the concepts to business problems

BOOK p.54 ↗

IMPORTANT

SEE Feb'26 3a (8M)

Question (Tables 3.2–3.4)	Fraud detection (Paul)	Tweeting Coke machines (Frank) — SEE Feb'26 3a	E-commerce recommendations (Rex)
Impact if collection↔analysis link down for long?	Catastrophic — service can't run	Lose money/offers; more if used for inventory management	Lost up-sell/cross-sell, lost sales
Days of data tolerable to lose?	Zero	Depends on inventory handling & machine popularity (locale-specific)	Maybe batch recommender as backup; freshness issue
Need historical data?	Yes — performance reports	Perhaps — reporting on machine performance	Yes — reporting, algorithm refinement
Delivery semantics?	Exactly-once (at-least-once possible, more consumer work)	At-least-once suffices (at-most-once arguable)	At-least-once (exactly-once overkill)

🔥 HOW TO WRITE IT — SEE Feb'26 3a — answer the 4 sub-questions in 2 marks each

Use the Frank column: (i) revenue lost / wrong offers / stale stock recommendation, (ii) tolerance small-to-moderate and locale-specific, (iii) yes for reporting/inventory trends, (iv) **at-least-once** with idempotent consumers, justified because a duplicate tweet/offer is harmless but a missed one costs a sale; use durable, replicated broker.

6 · Analysing streaming data — in-flight analysis

BOOK p.59 ↗

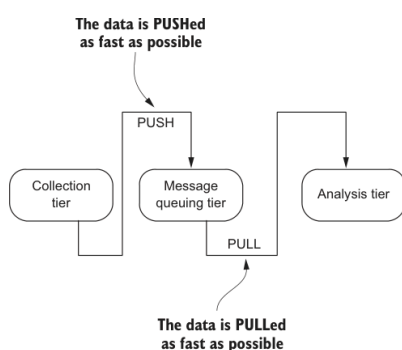


Figure 4.2 Data being pushed from the collection tier and pulled from the analysis tier

Fig 4.2 Push into queue, pull by analysis (p.60)

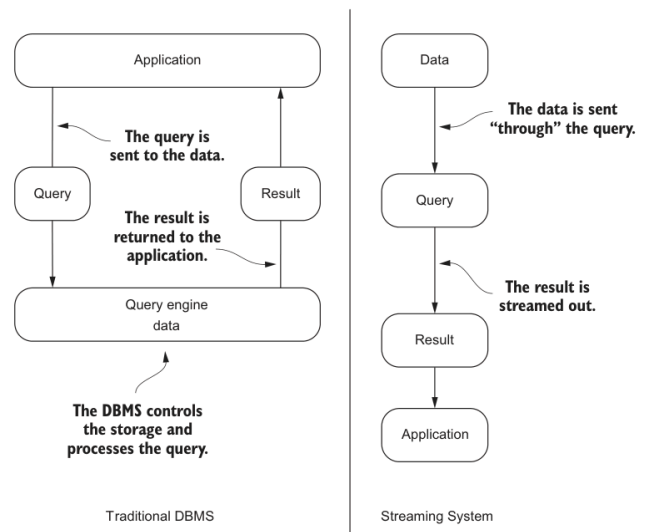


Figure 4.3 Turning things on their head: non-streaming versus streaming

Fig 4.3 Traditional DBMS vs streaming system (p.61)

- **In-flight data** = all tuples in the system from input (queue tier) to output (client): **always in motion, never at rest** (never persisted to durable storage).
- **Continuous query model**: instead of querying data at rest, **data is moved through the query**; a user registers a query once, it runs each time data arrives or at an interval, results are **pushed**. Example: trending article / broken link — 3 steps (collect, start query, act) vs 5 in a DBMS (gather, load, query, act, repeat every x hours). ([BOOK p.60 ↗](#))
- Inverts the model: DBMS user = **active**, system **passive**; streaming system = **active**, user **passive**.

Table 4.1 — DBMS vs streaming system ([BOOK p.62 ↗](#)) **IMPORTANT** CIE-2 Jan'25 1a (6M) · CIE-3 Jan'26 A2

	Traditional DBMS	Streaming system
Query model	One-time query on consistent state; answer returned, query forgotten; pull	Continuous query on flowing data; registered once, results regularly pushed
Changing data	During downtime data can't change	Data keeps arriving while analysis tier is down ⇒ catch-up after crash
Query state	Crash ⇒ query forgotten; user re-issues it	Registered queries may or may not resume where they left off — often as if never stopped
Data set	Static, loaded before query	Unbounded, never ends

Use-case categories (p.62): tracking behaviour (McDonald's NL: +700% offer redemptions), traffic safety/efficiency (EU congestion ~€100 bn/yr), real-time fraud analytics (US card fraud losses down 70% as a % of sales).

7 · Distributed stream-processing architecture ([BOOK p.63 ↗](#)) **IMPORTANT**

CIE-3 Jan'26 1a (10M) · Model QP 1.4 · CIE-2 Jan'25 A1

Need: volume/velocity exceeds one machine (a GE gas turbine makes ~1 TB/hour) ⇒ distributed analysis tier. Popular open-source engines: **Spark Streaming, Storm, Flink, Samza** (all Apache). All share three parts: a component the application is **submitted** to, **separate nodes** that run the algorithm, and **data sources** as input.

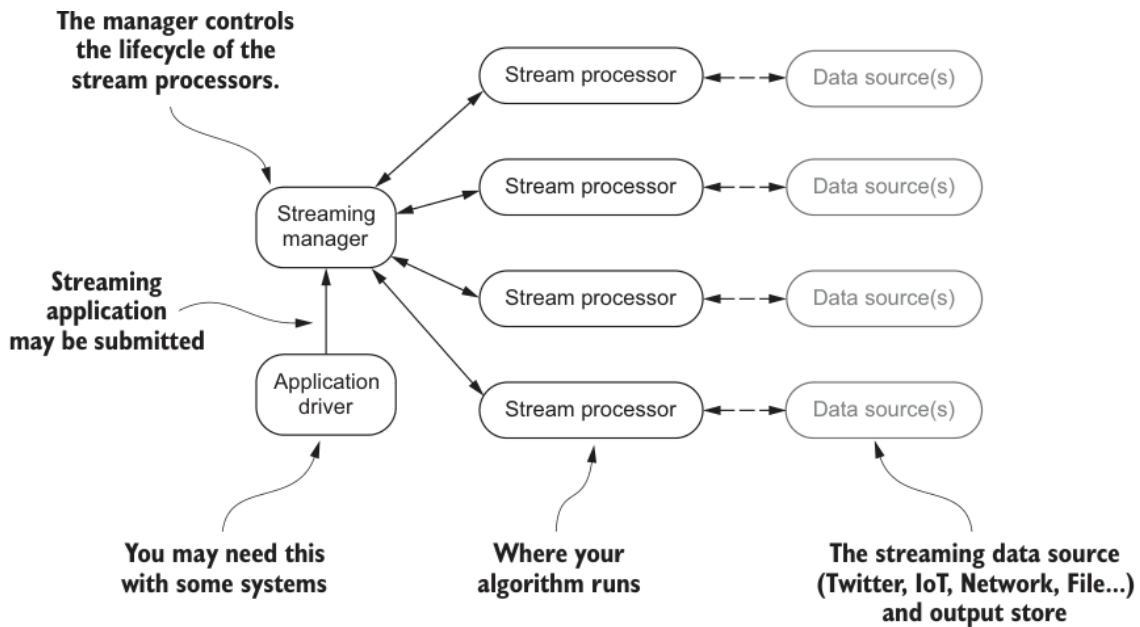


Figure 4.4 Generic streaming analysis architecture you will find with many products on the market

Fig 4.4 Generic streaming analysis architecture (p.64) — DRAW THIS

Component	Role
Application driver	Client code that defines the streaming program and talks to the manager (Spark: driver submits the job, may collect results, controls job lifetime). Needed in some systems only.
Streaming manager	Gets the job to the stream processors; controls their lifecycle; may request resources.
Stream processor	Where the job runs ("rubber meets the road").
Data source(s)	Input (and possibly output store) — Twitter, IoT, network, files. Some platforms ingest many sources per job, others one.

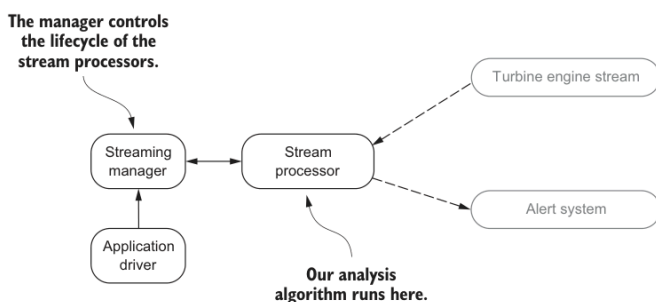


Figure 4.5 Turbine engine monitoring on our common architecture

Fig 4.5 Turbine monitoring mapped to the common architecture (p.65)

7.1 The four frameworks BOOK p.65 ↗

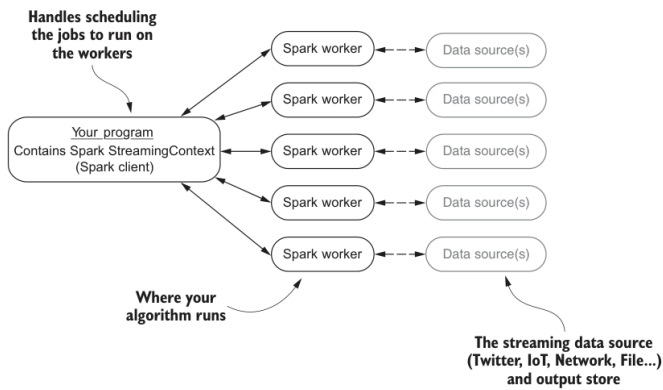


Figure 4.7 Spark Streaming's high-level architecture

Fig 4.7 Spark Streaming (p.66)

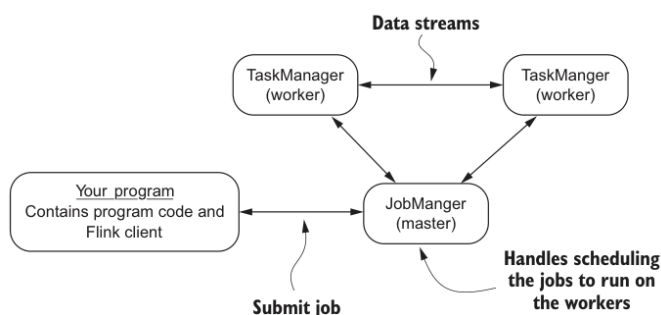


Figure 4.9 High-level Apache Flink architecture

Fig 4.9 Flink (p.68) — SEE Feb'26 Q1.4 "diagrammatically represent Flink"

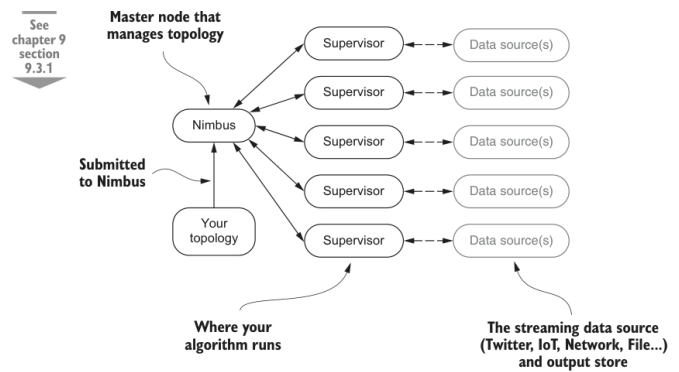


Figure 4.8 High-level overview of Storm architecture showing Nimbus, supervisors, and a data source

Fig 4.8 Storm (p.67) – SEE Q1.4 asks Flink, Model QP Q1.4 asks Storm

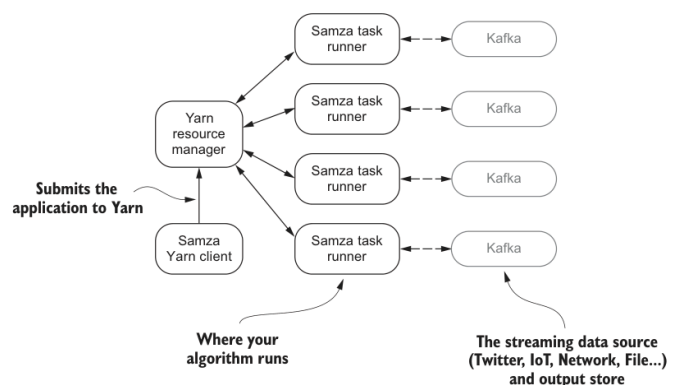


Figure 4.10 High-level Apache Samza architecture

Fig 4.10 Samza (p.68)

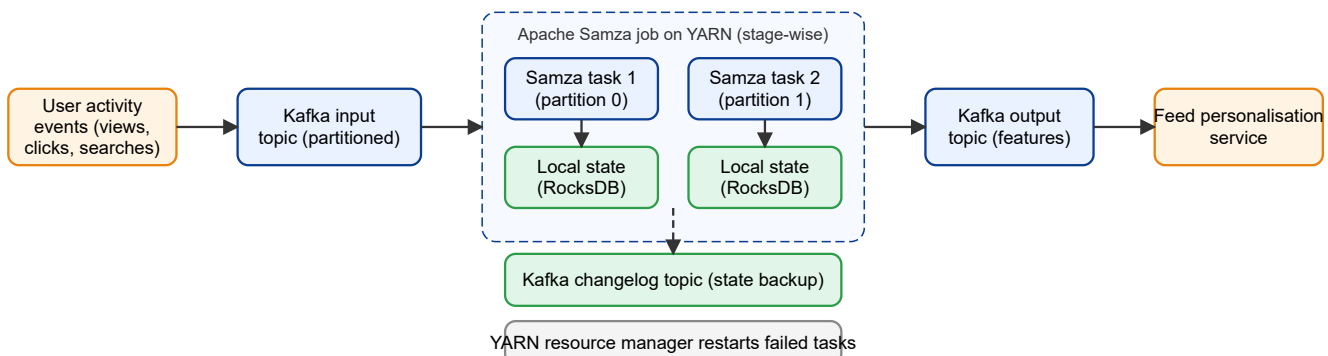
	Spark Streaming	Storm	Flink	Samza
Model	Micro-batches (jobs over short time slices, even <0.5 s)	Tuple-at-a-time	Stream-first : everything is a stream; streams + transformations; dataflow source→transform→sink	Stage-wise stream processing
Unit of work	Job	Topology (runs forever)	Streaming dataflow / job	Samza job/tasks
Master / worker	Driver with StreamingContext → Spark workers	Nimbus (master: deploys topology, assigns tasks, monitors failures) → Supervisors	JobManager (master) → TaskManagers (workers); Flink client submits job	YARN resource manager + Samza task runners ; Samza YARN client submits
Built on / needs	Apache Spark (MLlib, GraphX, SparkSQL; Java/Scala/Python/R)	—	—	Apache YARN (resources, scheduling) + Apache Kafka (all input/output)

🧠 **MNEMONIC — "S-S-F-S": Spark = Small batches, Storm = Single tuple, Flink = Stream-first, Samza = Stage-wise (YARN+Kafka)**

7.2 Samza scenario — professional networking platform (SEE Feb'26 4a, 10M)

IMPORTANT

Question: process millions of user-activity events/sec (profile views, clicks, job searches, feed interactions) in near real time; goals low-latency personalisation, high throughput + scalability, strong fault tolerance + state consistency. "Implement Apache Samza architecture, draw diagram, explain steps." (Answer built from Ch.4 Samza text + Ch.4 state/fault-tolerance features; the LinkedIn scenario itself is not in the textbook.)



Samza architecture for the activity-feed scenario (redrawn — our diagram)

1. Activity events are written to **Kafka** (input topics partitioned by member ID) — Samza reads **and writes all data through Kafka**.
2. Samza job submitted by the **Samza YARN client** to the **YARN resource manager**, which allocates containers and starts **Samza task runners** (one task per input partition) ⇒ **horizontal scalability** and high throughput.
3. Each task keeps **local state** (e.g. member's recent views/clicks) for low-latency personalisation features; state change log is written to a Kafka changelog topic ⇒ **replicated, recoverable state** (state-management continuum, §8).
4. Results published to an **output Kafka topic** (chaining stream jobs — Samza's core pattern) consumed by the feed-personalisation service.
5. **Fault tolerance**: a failed task is restarted by YARN and rebuilds state by replaying the changelog from its last checkpointed Kafka offset (rollback recovery, §9); Kafka replication protects the data.
6. **Delivery semantics**: at-least-once with idempotent updates (or exactly-once if the framework supports it) so duplicates do not corrupt counts ⇒ consistent state.

8 · Key features of stream-processing frameworks **BOOK p.69 ↗**

8.1 Message delivery semantics in the analysis tier **BOOK p.69 ↗**

- **At-most-once** — a message may be lost but never processed twice; simplest, no special logic. **At-least-once** — never lost, may be processed more than once: the manager tracks

sends + acks and **re-sends** on timeout ⇒ the job must be **idempotent** (same message ⇒ same result). **Exactly-once** — also detects and ignores duplicates; most complex.

- **Choosing:** turbine-monitoring prediction may tolerate missing data ⇒ least complex guarantee first; **ad network billing** ⇒ exactly-once. Always design jobs to be idempotent.

[BOOK p.71 ↗](#)

- CIE-2 Jan'25 A2 "Storm vs Samza message semantics" — see Storm/Samza text above: Storm tuple-at-a-time, Samza stage-wise via Kafka; the book does not tabulate their guarantees, so answer with the three definitions + say Samza relies on Kafka's log for replay.

⚠ CONFLICT / TRAP — Figure 4.11 caption error

The book's Fig 4.11 (p.70) is captioned "At-least-once" but it illustrates **at-most-once** (messages dropped / processor crashed, "two messages went in and only one came out"). In the exam, label that diagram at-most-once.

8.2 State management [BOOK p.71 ↗](#) **IMPORTANT** CIE-3 Jan'26 2a (4M) · Model QP 1.10 (internal vs external state, Unit 5 too)

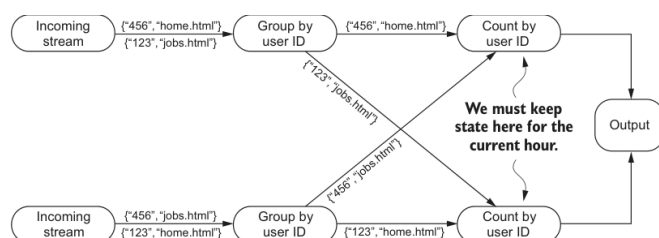


Figure 4.12 Simple example of counting page views per user over an hour

Fig 4.12 Page views per user per hour: state kept in the counter (p.72)

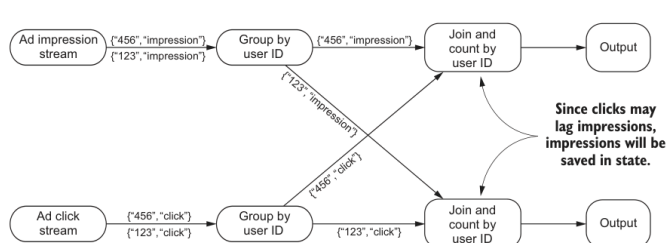


Figure 4.14 Handling ad impression and ad click streams that use stream state

Fig 4.14 Ad impression + ad click streams joined using state (p.73)

- State is needed once an algorithm depends on **previous messages or external data** (counts, joins, late events). Without state management, a failure loses intermediate results.
- **Continuum** (Fig 4.13): **in-memory only** (low complexity; lose everything on failure — keep in memory and flush each hour) → ... → **replicated, queryable, persistent store** (high complexity; allows joins of streams). Low end only keeps a computation alive through failures; high end answers complex questions.
- **Example (CIE-3 scheme):** ads — clicks lag impressions, so impressions are **retained in state keyed by user/ad ID** until the matching click arrives; persistent + replicated state survives failures and enables the join and click-through counts.

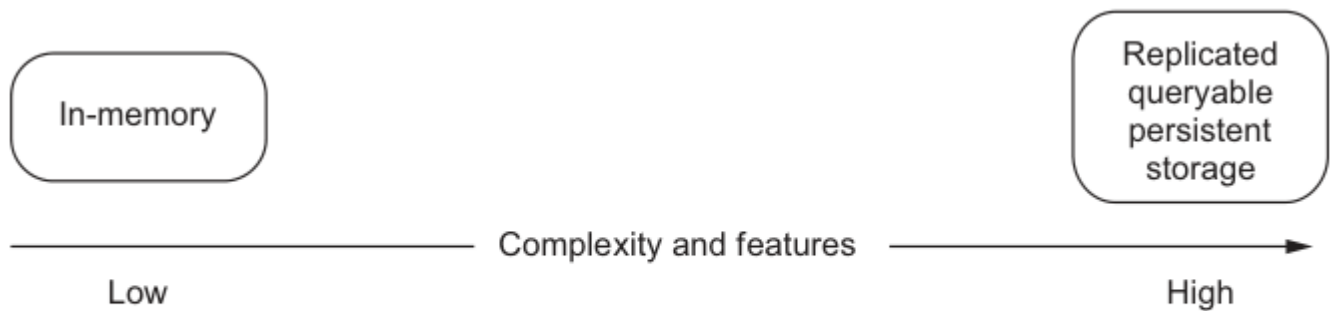


Figure 4.13 State management complexity continuum for stream-processing tools

Fig 4.13 State-management complexity continuum (p.72)

8.3 Fault tolerance — the SEVEN points of failure [BOOK p.73 ↗](#) **IMPORTANT** Model QP 4a (10M) · CIE-2 Jan'25 3a (10M) · SEE Feb'26 8b

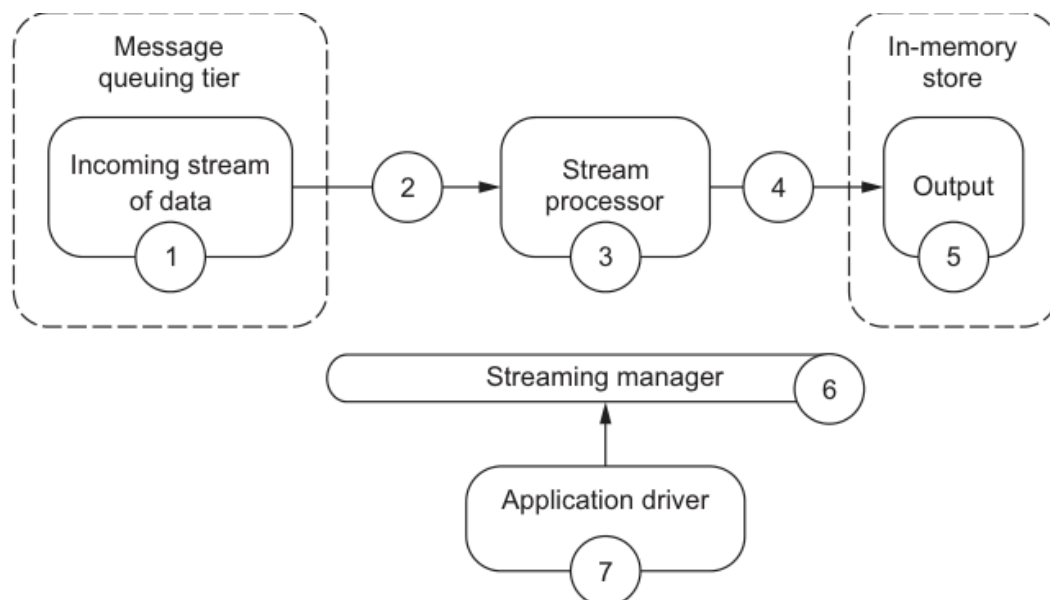


Figure 4.15 The points of failure with stream processing in the context of the streaming architecture

Fig 4.15 Points of failure with stream processing in the streaming architecture (p.74) — DRAW THIS

#	Failure point	What the framework must do
1	Incoming stream of data (queue tier may fail)	Respond gracefully; don't fail if data/resource unavailable
2	Network carrying the input stream	Handle disruption gracefully (can't control it)
3	Stream processor (job software or machine fails)	Streaming manager restarts it or moves it to another machine
4	Connection to output destination	Control flow from last processor; avoid network back-pressure overload
5	Output destination	Not supervised, but its failure impacts processing
6	Streaming manager	If it fails the processors run headless — no supervision, no new/recovered processors
7	Application driver	Only a risk if it contains the manager (second flavour)

- Consolidated: **data loss** (network, processor crash with in-memory data) and **loss of resource management** (manager, driver). Techniques are all **replication + coordination**; **k-fault tolerant** = survives k simultaneous failures.

🗨️ MNEMONIC — I-N-P-C-O-M-D

Incoming stream · Network · Processor · Connection to output · Output destination · Manager · Driver.

State-machine vs rollback recovery BOOK p.75 > IMPORTANT CIE-3 Jan'26 3b (4M) · CIE-2 Jan'25 A3

	State-machine approach	Rollback recovery
Idea	Manager replicates the job on independent nodes , sends the same input in the same order to all	Processor periodically checkpoints state (to another node/disk, often with in-memory logs); on failure rebuild from last checkpoint and replay the log
Cost	k + 1 times the resources of one replica (high)	Lower overhead
Recovery	Quick failover , little disruption	Slower recovery (checkpoint + replay); disks add latency
Best for	Low-latency always (e.g. intrusion detection)	Fault tolerance important, rare moderate latency acceptable

Both assume the job is **idempotent and consistent**: two non-faulty processors with the same input in the same order give the same output in the same order. Fault tolerance via state machine in CIE-2 Jan'25 A3 (2M): replicate the job on k+1 nodes with identical ordered input, so another replica takes over instantly.

9 · Quick revision — Unit 2

Three parts of a message queuing system?	Producer, broker, consumer.
Why "broker" not "queue"?	Broker abstracts over multiple queues.
Two things the queue tier gives (Part A)?	Decoupling, buffering of surges, durability, delivery guarantees.
Durable vs transient?	Durable persisted (JDBC/file/proprietary); transient deleted after consumption.
Three semantics?	At-most, at-least, exactly-once.
Two techniques for exactly-once without product support?	Don't blindly retry (verify with broker); store last-message metadata (Kafka offset).
Two bonuses of those techniques?	Message auditing, duplicate detection.
Broker crash mitigations?	Producer ack after disk write; replicate to >1 broker; minimal in-memory data.
In-flight data means?	Tuples between input and output, never at rest.
Continuous query = ?	Registered once, evaluated on arriving data, results pushed.
Storm vs Samza?	Storm tuple-at-a-time (Nimbus/supervisors); Samza stage-wise on YARN + Kafka.
Spark Streaming processes?	Micro-batches (<0.5 s).
Flink master and workers?	JobManager, TaskManagers.
Idempotent job?	Same input $\Rightarrow$ same output; needed for at-least-once.
Points of failure in stream processing?	7 (incoming stream, network, processor, output connection, output, manager, driver).
"Running headless"?	Streaming manager down; processors keep running unsupervised.
State-machine cost?	$k+1$ times resources.